LAB7 예비 보고서

STM32_3 SPI — MCU·FPGA 사이의 송수신

과목 전자전기컴퓨터설계실험 2

제출일 2026년 11월 2일

성명 김태이

학번 2023440158

1 보고서 범위와 결론

11월 2일 실험(STM32_3)은 SPI 하나에 집중한다. MCU의 SPI1로 한 바이트를 보내 파형을 보고(과제8), MCU 두 대 사이에서 주고받은 뒤(과제9), FPGA에 SPI 수신 회로(과제10)와 송신 회로(과제11)를 직접 만들어 MCU와 잇고, 마지막으로 FPGA 두 대를 연결한다(과제12). 형식은 Mode 0, 8비트, 높은 비트 먼저이고 MISO는 쓰지 않는다. FPGA Verilog는 제공되지 않으므로 수신·송신 회로를 처음부터 설계했다.

MCU 코드는 한 파일에 쓰고 매크로 STM32_3_ROLE로 송신(마스터)과 수신(슬레이브)을 골라 Arm GCC로 빌드했다. 같은 소스를 PC에서 실행하는 호스트 시뮬레이션이 송신 바이트를 파일로 남기면 FPGA 수신 테스트벤치가 그 파일을 MCU와 같은 타이밍의 SPI 신호로 넣고, 반대로 FPGA 송신 테스트벤치가 MOSI에서 읽은 바이트를 MCU 수신 코드가 받는다. 네 가지 연결(과제9~12)을 모두 코드 수준에서 이어 본 셈이다.

FPGA는 수신 회로(spi_rx_uart)와 송신 회로(spi_tx_uart) 두 bitstream을 만들었다. 둘 다 50 MHz에서 타이밍 여유가 14 ns 이상이다. FPGA-FPGA 연결은 두 보드의 클록을 일부러 다르게 두고(0.25 % 차이) 선 지연을 넣은 테스트벤치로 확인했다.

과제	사전 검증	결과
8	Arm GCC 빌드, 호스트 8바이트	55 A5 00 FF, 비트 순서 일치
9	호스트 마스터 → 슬레이브	RX 8줄 일치, 타임아웃 문구
10	MCU 파일 → FPGA TB	UART 9줄·LED 일치, 끊긴 바이트 복구
11	FPGA TB → MCU 호스트	SCK 500 kHz, RX 8줄 일치
12	보드 두 대 TB	클록이 달라도 8바이트 일치
구현	두 설계 bitstream	WNS +14.349 / +15.484 ns

PASS와 호스트 시뮬레이션 결과는 논리가 맞다는 뜻이다. 실제 선의 파형(울림, 지연), MCU HAL 함수가 CS와 SCK 사이에 넣는 시간, 점퍼선 결선, 보드 클록 스위치 상태는 포함하지 않으므로 각 장 끝의 표대로 오실로스코프와 보드에서 확인한다.

2 실험 환경과 검증 방법

구분	사용한 것
MCU 보드	NUCLEO-G474RE (STM32G474RE, Cortex-M4)
MCU 프로젝트	Glaysia/ece2_26_2_2의 c/stm32/base
MCU 빌드	Arm GNU Toolchain 13.3.Rel1, CMake 3.24.2, Ninja 1.11.1
호스트 검증	MinGW-W64 gcc 9.5.0, HAL 대역 헤더
FPGA 보드	Combo II-DLD, xc7s75fgga484-1
FPGA 도구	Vivado 2025.2, Icarus Verilog 12.0
계측	오실로스코프(SCK·MOSI·CS), Serial Monitor 2개

조교 저장소가 이번 주에 ece2_26_2_2로 옮겨졌다. 새 저장소의 기본 프로젝트(c/stm32/base)는 LAB6까지 쓴 프로젝트와 파일이 같고, ece2_474_routine.c도 원본과 한 글자도 다르지 않음을 diff로 확인했다. 그래서 LAB6 폴더를 lab7/stm32_lab으로 복사한 뒤 ece2_474_routine.c만 원본으로 되돌리고 이번 과제를 작성했다. 매뉴얼의 SPI 제공 코드 파일(08_spi_master, 09_spi_slave)은 stm32_lab/docs/examples에 함께 두었다.

빌드는 세 가지다. 기본(Debug 프리셋)은 송신 측으로 0x55, 0xA5, 0x00, 0xFF를 1초마다 차례로 보낸다. -DSTM32_3_TX_FIXED=0x55를 주면 한 값만 계속 보내 오실로스코프로 보기 쉽고, -DSTM32_3_ROLE=2를 주면 수신 측이 된다.

호스트 시뮬레이션은 LAB6의 HAL 대역에 SPI 함수(Init, DeInit, Transmit, Receive)와 GPIO_Init을 더했다. HAL_GetTick은 1 ms씩 늘어나는 가상 시각이다. 송신 대역은 보낸 바이트를 파일에 적고, 같은 바이트를 Mode 0 규칙대로 SCK·MOSI·CS 시각표(CSV)로 펼친 뒤 상승 에지에서 다시 읽어 원래 값이 나오는지 확인한다. 수신 대역은 init이 SPI1을 슬레이브·NSS 하드웨어 입력으로 바꾸고 PA4를 AF5로 설정했을 때만 파일의 바이트를 돌려준다.

FPGA는 프로젝트마다 기능 검사용 tb.sv와 파형 확인용 tb_view.sv를 직접 작성해 Icarus와 Vivado 시뮬레이터에서 실행했다. Vivado 파형 캡처는 한 단 폭에서 읽히도록 시간축을 반으로 잘라 두 줄로 배치했다.

3 소스 출처와 실행 스냅샷

항목	식별 정보
MCU 템플릿	https://github.com/Glaysia/ece2_26_2_2.git (c/stm32/base)
커밋	1fab83ee3c6bb595c73d7834be143d3865efa92b
FPGA 템플릿	fpga-lab-template v2.0.2 (같은 저장소 templates/fpga)
FPGA 코드	직접 작성. uart_tx만 LAB3-07(lab6_04_uart_led)과 같음
제출 저장소	https://github.com/tae2yi/UOS_ece2
LAB7 코드	19a037b (보고서의 GitHub 주소가 가리키는 위치)
폴더	lab7/stm32_lab, stm32_3_hostsim, lab7_01_spi_rx, lab7_02_spi_tx, lab7_03_spi_link

GitHub: https://github.com/tae2yi/UOS_ece2/tree/19a037b/lab7

템플릿에서 바꾼 MCU 파일은 Core/Src/ece2_474_routine.c 하나다. 원본 대비 바뀐 내용 전체는 아래 diff에 있다. 호스트 시뮬레이션의 입력·출력, FPGA 시뮬레이션 로그, Vivado 리포트도 같은 저장소에 있다.

diff:
https://github.com/tae2yi/UOS_ece2/blob/19a037b/lab7/report/pre/logs/stm32_routine.diff

STM32_3 SPI 통신

신호와 Mode 0

SPI는 클록 선을 따로 두는 동기식 직렬 통신이다. 마스터가 SCK를 만들고 MOSI로 데이터를 내보내며, CS(칩 선택)를 LOW로 내려 통신할 슬레이브를 고른다. 슬레이브는 받은 SCK의 에지에 맞춰 MOSI를 읽으므로, UART처럼 양쪽이 속도를 미리 약속할 필요가 없다. 대신 선이 하나 더 필요하다[2]. 슬레이브가 마스터로 보내는 MISO 선도 있지만 이번 실습은 한 방향씩만 보므로 연결하지 않는다.

Mode는 클록 극성(CPOL)과 위상(CPHA)의 조합이다. Mode 0(CPOL=0, CPHA=0)은 SCK가 쉬는 동안 LOW이고, 첫 에지인 상승 에지에서 읽는다. 따라서 보내는 쪽은 첫 상승 에지 전에 첫 비트(bit7)를 내놓고, 하강 에지마다 다음 비트로 바꾼다. 읽는 순간에 MOSI가 반주기 동안 안정되어 있는 구조다. main.c의 설정이 이와 같다(CLKPolarity LOW, CLKPhase 1EDGE, FirstBit MSB).

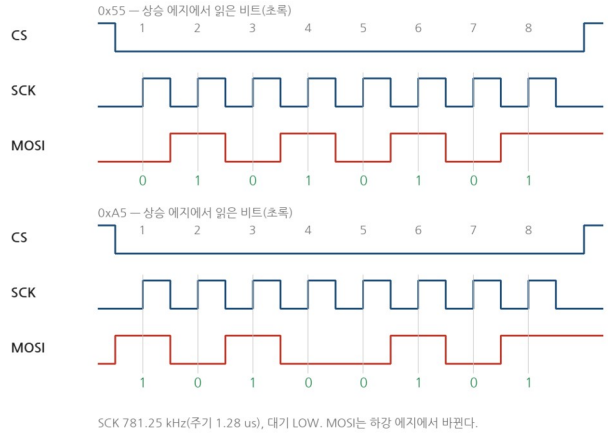


그림 1. MCU 호스트 시뮬레이션이 만든 0x55·0xA5의 SCK·MOSI·CS 시각표. 초록 숫자는 상승 에지에서 읽은 비트다.

0x55를 높은 비트부터 읽으면 0 1 0 1 0 1 0 1, 0xA5는 1 0 1 0 0 1 0 1이다. 0x55는 비트가 매번 바뀌므로 MOSI가 SCK의 절반 주파수로 토글하고, 0x00과 0xFF는 MOSI가 내내 LOW 또는 HIGH라 비트 경계가 보이지 않는다. 이때는 SCK 상승 에지 8번으로 바이트 길이를 확인한다.

SCK 주파수

SPI1은 APB2 클록을 쓴다. main.c에서 HSI 16 MHz를 PLL로 ÷2 ×25 ÷2 하여 시스템 클록 100 MHz를 만들고 APB2 분주는 1이다. 분주비가 128이므로 SCK = 100 MHz / 128 = 781.25 kHz, 주기 1.28 µs이고 한 바이트(8비트)는 10.24 µs다. FPGA 마스터는 50 MHz를 100으로 나눠 500 kHz(주기 2 µs)로 만들어 매뉴얼의 1 MHz 이하 조건을 지킨다.

쪽	분주	SCK	1바이트
MCU SPI1 마스터	100 MHz / 128	781.25 kHz	10.24 µs
FPGA spi_master_tx	50 MHz / (2×50)	500 kHz	16 µs

코드 읽기 항목

표현	뜻
SPI_HandleTypeDef	SPI 설정·상태를 묶은 구조체. hspi1은 그 변수
hspi1.Init.Mode	구조체 안의 설정 항목. 점(.)으로 접근
HAL_SPI_Transmit(&hspi1, &tx, 1U, 100U)	1바이트 송신, 최대 100 ms 대기
HAL_SPI_Receive(…, 2000U)	1바이트 수신, 최대 2 s 대기
SPI_NSS_HARD_INPUT	NSS(PA4) 핀으로 선택 신호를 받음
GPIO_AF5_SPI1	PA4를 SPI1의 대체 기능으로 연결
%02X	최소 두 자리 대문자 16진수

HAL_SPI_Transmit은 바이트를 송신 FIFO에 넣고 전송이 끝날 때까지 기다린 뒤 돌아온다[1]. 결과 HAL_OK(0)는 MCU 쪽 송신이 끝났다는 뜻일 뿐이고, MISO를 쓰지 않으므로 상대가 받았는지는 알 수 없다. 받았는지는 수신 측의 출력으로만 확인한다.

전기적 연결과 주의

STM32G474와 Combo II-DLD의 FPGA I/O는 모두 3.3 V 논리라 점퍼선으로 바로 잇는다. FPGA 쪽은 LAB6과 같이 브레드보드 I/O의 BDIO_0~2(F8, E8, A7)를 SCK, MOSI, CS에 쓰고 GND는 옆 헤더 J6에 잇는다. 이 핀들은 버스 스위치와 100 Ω 직렬 저항을 거치며 DIP 스위치 SW5(BD_SEL)를 켜야 FPGA와 이어진다. 두 장치의 GND는 반드시 함께 묶는다.

한 선에 출력이 둘 이상 붙지 않게 하는 것이 가장 중요하다. MCU가 송신 프로그램인 채로 FPGA 송신 회로에 연결하면 PB3·PA7과 F8·E8이 서로 반대 값을 낼 수 있다. 그래서 받는 쪽에 수신 프로그램(또는 수신 bitstream)을 먼저 올린 다음 선을 잇고, 역할을 바꿀 때도 프로그램을 먼저 바꾼다.

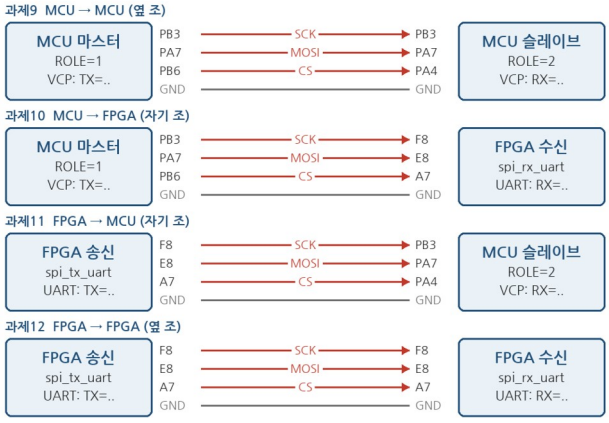


그림 2. 과제9~12의 연결. 왼쪽이 송신(마스터), 오른쪽이 수신이다. MCU 수신 측의 CS는 PB6가 아니라 PA4(NSS)로 받는다.

STM32_3 과제8 MCU 송신 파형 확인

요구 조건과 설계

- 1초마다 한 바이트를 SPI1로 보내고, 보낸 값과 결과를 VCP UART에 출력한다.
- 오실로스코프로 SCK·MOSI·CS를 보고 비트 순서와 SCK 주파수를 기록한다.
- 보내는 값을 0x55, 0xA5, 0x00, 0xFF로 바꿔 비교한다.

매뉴얼의 송신 예제는 0x55 한 값만 보낸다. 값을 바꿀 때마다 다시 빌드하지 않도록 네 값을 배열에 두고 1초마다 차례로 보내게 했다. 한 값만 계속 봐야 할 때는 STM32_3_TX_FIXED 매크로로 고정한다. 1초 간격은 HAL_GetTick()의 차이로 재므로 polling_routine이 기다리지 않고 바로 돌아간다. CS(PB6)는 main.c가 처음에 HIGH로 두고, 보낼 때만 LOW로 내렸다가 올린다.

핵심 코드

```
void polling_routine(void)
{
#if STM32_3_ROLE == 1
  /* 송신: 1초마다 CS LOW, 1바이트, CS HIGH */
  static uint32_t last_ms = 0;
  static uint32_t index = 0;
  uint32_t now_ms = HAL_GetTick();
  if (now_ms - last_ms < 1000U) return;
  last_ms = now_ms;


#ifdef STM32_3_TX_FIXED
  uint8_t tx = (uint8_t)(STM32_3_TX_FIXED);
  (void)index;
#else
  uint8_t tx = tx_values_ece2[index];
  index = (index + 1U) %
      (sizeof(tx_values_ece2) / sizeof(tx_values_ece2[0]));
#endif
  HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port,
           SPI_CS_ece2_Pin, GPIO_PIN_RESET);
  HAL_StatusTypeDef result =
    HAL_SPI_Transmit(&hspi1, &tx, 1U, 100U);
  HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port,
           SPI_CS_ece2_Pin, GPIO_PIN_SET);


  printf("TX=%02X, result=%d\r\n",
     (unsigned int)tx, (int)result);
#else
  /* 수신: 최대 2초 기다려 결과를 VCP로 출력 */
  uint8_t rx = 0;
  HAL_StatusTypeDef result =
    HAL_SPI_Receive(&hspi1, &rx, 1U, 2000U);
  if (result == HAL_OK)
  {
    printf("RX=%02X\r\n", (unsigned int)rx);
  }
  else if (result == HAL_TIMEOUT)
  {
    /* 매뉴얼 한 줄을 들로 나눔(같은 문자열) */
    printf("RX timeout: check sender, SCK, MOSI, "
       "CS and GND\r\n");
  }
  else
  {
    printf("RX error=%d\r\n", (int)result);
  }
#endif
}
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/19a037b/lab7/stm32_lab/Core/Src/ece2_474_routine.c#L91-L138

#if STM32_3_ROLE == 1 쪽이 송신이다. 마지막 바이트를 보낸 뒤 index가 0으로 돌아가도록 나머지 연산을 쓰고, 배열 길이는 sizeof로 구해 값을 늘려도 코드를 고칠 필요가 없게 했다. ece2_474_init()은 송신 측에서 비워 둔다. 기본 SPI1 설정이 이미 마스터·Mode 0이기 때문이다.

예상 결과

호스트 시뮬레이션으로 8.5초 동안 실행한 VCP 출력과, 같은 바이트를 시각표로 펼쳐 상승 에지에서 다시 읽은 결과다.

```
$ master 8   (VCP 출력)
TX=55, result=0
TX=A5, result=0
TX=00, result=0
TX=FF, result=0
TX=55, result=0
TX=A5, result=0
TX=00, result=0
TX=FF, result=0
$ 시각표 확인(앞 4바이트)
t= 1000 ms  SPI TX 55 01010101  -> 55 OK
t= 2000 ms  SPI TX A5 10100101  -> A5 OK
t= 3000 ms  SPI TX 00 00000000  -> 00 OK
t= 4000 ms  SPI TX FF 11111111  -> FF OK
```

결과:
https://github.com/tae2yi/UOS_ece2/tree/19a037b/lab7/stm32_3_hostsim/results

값	MOSI(상승 에지 순서)	오실로스코프에서
0x55	0 1 0 1 0 1 0 1	SCK 절반 주파수로 토글
0xA5	1 0 1 0 0 1 0 1	가운데 두 비트 0 0
0x00	0 0 0 0 0 0 0 0	MOSI LOW 고정
0xFF	1 1 1 1 1 1 1 1	MOSI HIGH 고정

CS가 LOW인 시간은 8비트 10.24 µs에 HAL 함수의 처리 시간이 더해진 값이다. HAL_GPIO_WritePin과 HAL_SPI_Transmit 진입 사이, 전송이 끝난 뒤 CS를 올리기까지 각각 수 µs가 걸릴 것으로 보지만 정확한 값은 코드만으로 정할 수 없어 오실로스코프로 잰다.

빌드 결과와 보드에서 확인할 항목

```
$ cmake --preset Debug && cmake --build --preset Debug
RAM:    2648 B / 128 KB  (2.02 %)
FLASH: 28924 B / 512 KB  (5.52 %)
$ ... -DSTM32_3_TX_FIXED=0x55    FLASH 28892 B
$ ... -DSTM32_3_ROLE=2           FLASH 30764 B
경고 0개 (세 빌드 모두)
```

로그:
https://github.com/tae2yi/UOS_ece2/blob/19a037b/lab7/report/pre/logs/stm32_arm_build.txt

확인 항목	예상 관찰
VCP(9600)	1초마다 TX=55, result=0 → A5 → 00 → FF
CS(PB6)	평소 HIGH, 1초마다 약 10 µs 이상 LOW
SCK(PB3)	CS LOW 동안 상승 에지 8번, 대기 LOW
SCK 주기	1.28 µs = 781.25 kHz
MOSI(PA7)	위 표의 비트열, 하강 에지에서 바뀜
CS↓ → 첫 SCK↑	시간 기록(HAL 처리 시간)

STM32_3 과제9 MCU → MCU 수신 확인

수신 측 설정

수신 MCU는 main.c와 CubeMX를 고치지 않고 ece2_474_init()에서 SPI1을 다시 설정한다. 먼저 HAL_SPI_DeInit으로 마스터 설정을 풀고, Mode를 슬레이브로, NSS를 하드웨어 입력으로 바꿔 다시 초기화한 뒤 PA4를 SPI1의 NSS 기능(AF5)으로 연결한다. 송신 측의 CS는 일반 GPIO PB6이지만 수신 측은 SPI 하드웨어가 직접 읽는 PA4로 받아야 한다. 풀업을 켜서 선이 빠져도 선택되지 않은 상태(HIGH)로 둔다.

```
#if STM32_3_ROLE == 2
  /* 수신: main.c는 그대로, SPI1만 슬레이브로 재설정 */
  if (HAL_SPI_DeInit(&hspi1) != HAL_OK)
    Error_Handler();


  hspi1.Init.Mode = SPI_MODE_SLAVE;
  hspi1.Init.NSS = SPI_NSS_HARD_INPUT;
  if (HAL_SPI_Init(&hspi1) != HAL_OK)
    Error_Handler();


  GPIO_InitTypeDef gpio = {0};
  gpio.Pin = SPI_NSS_RX_ece2_Pin;
  gpio.Mode = GPIO_MODE_AF_PP;
  gpio.Pull = GPIO_PULLUP;
  gpio.Speed = GPIO_SPEED_FREQ_LOW;
  gpio.Alternate = GPIO_AF5_SPI1;
  HAL_GPIO_Init(SPI_NSS_RX_ece2_GPIO_Port, &gpio);
#endif
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/19a037b/lab7/stm32_lab/Core/Src/ece2_474_routine.c#L41-L58

수신 측 polling_routine은 앞 코드의 #else 쪽이다. HAL_SPI_Receive로 최대 2초 기다리고, 받으면 RX= 값, 시간이 지나면 배선을 확인하라는 문구를 출력한다. 송신 측이 1초마다 보내므로 정상이면 타임아웃이 나오지 않는다.

예상 결과

송신 측 호스트 시뮬레이션이 보낸 8바이트를 수신 측에 넣은 결과와, 중간에 2초 동안 아무것도 오지 않은 경우의 결과다.

```
$ slave < master_spi.hex
RX=55
RX=A5
RX=00
RX=FF
RX=55
RX=A5
RX=00
RX=FF
$ slave < 55, (2초 없음), A5
RX=55
RX timeout: check sender, SCK, MOSI, CS and GND
RX=A5
```

결과:
https://github.com/tae2yi/UOS_ece2/tree/19a037b/lab7/stm32_3_hostsim/results

수신 측을 먼저 실행해야 한다. 송신 측이 바이트를 보내는 도중에 수신 측이 시작하면 첫 바이트의 일부 비트를 놓쳐 값이 밀릴 수 있다(예: 0x55가 0xAA처럼 보임). 그런 값이 나오면 송신 측을 멈추고 수신 측을 리셋한 뒤 다시 시작한다.

역할 교대와 보드에서 확인할 항목

앞 조가 송신·뒤 조가 수신으로 먼저 하고 바꾼다. 바꿀 때는 프로그램만이 아니라 CS 선도 새 송신 측의 PB6 → 새 수신 측의 PA4로 옮긴다. SCK(PB3)와 MOSI(PA7)는 양쪽이 같은 핀이라 그대로 둔다.

확인 항목	예상 관찰
수신 측만 실행	2초마다 RX timeout 문구(정상 대기)
송신 측 실행	수신 VCP에 RX=55, A5, 00, FF 반복
송신·수신 출력 비교	같은 순서, 같은 값
GND 선 제거	값이 틀리거나 타임아웃(비교용)
역할 교대	CS 선 PB6 → PA4 다시 연결 후 같은 결과

STM32_3 과제10 MCU → FPGA 수신 구현

목적과 원리

FPGA가 SPI 슬레이브가 되어 MCU가 보낸 바이트를 받고, 그 값을 보드의 USB-UART로 PC에 보내 MCU의 송신값과 비교한다. FPGA 안에는 50 MHz 클록이 따로 있으므로 SCK를 클록으로 쓰지 않고 입력 신호로 취급한다. SCK·MOSI·CS를 50 MHz 클록에 맞춰 받은 뒤, SCK의 이전 값과 현재 값을 비교해 상승 에지를 찾는다.

외부 신호는 FPGA 클록과 관계없이 바뀌므로 플립플롭의 준비 시간을 어길 수 있고, 그러면 출력이 잠시 0도 1도 아닌 상태(메타안정)에 머문다. 플립플롭 두 개를 이어 한 클록 동안 안정될 시간을 주는 것이 2단 동기화다 [2]. 세 신호가 같은 2클록 지연을 거치므로 동기화된 SCK의 상승 에지에서 읽은 MOSI는 실제 상승 에지 때의 값과 같다.

MCU의 SCK 반주기는 640 ns = 32클록이고 MOSI는 하강 에지에서만 바뀐다. 동기화와 에지 검출에 걸리는 3클록(60 ns)은 반주기에 비해 매우 짧아 읽는 순간 MOSI가 바뀔 위험이 없다. 이 방식은 SCK의 HIGH·LOW가 각각 2클록 이상이어야 하므로 이론상 SCK 12.5 MHz 미만에서 동작하며, 이번 0.78 MHz와 0.5 MHz는 여유가 크다.

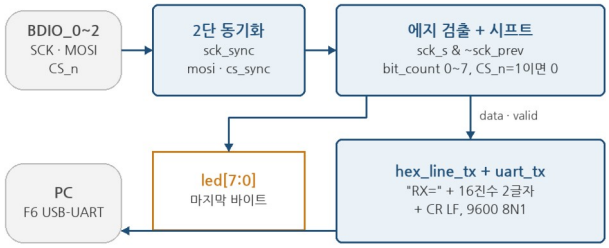


그림 3. FPGA 수신 회로(spi_rx_uart). 동기화 → 에지 검출·시프트 → 바이트 완성 → UART 한 줄 출력과 LED.

핵심 코드

```
always @(posedge clk) begin
  if (rst_p) begin
    sck_sync <= 2'b00; mosi_sync <= 2'b00; cs_sync <= 2'b11;
    sck_prev <= 0; shift <= 0; bit_count <= 0;
    data <= 0; valid <= 0;
  end else begin
    sck_sync  <= {sck_sync[0], sck};
    mosi_sync <= {mosi_sync[0], mosi};
    cs_sync   <= {cs_sync[0], cs_n};
    sck_prev  <= sck_s;
    valid <= 0;
    if (cs_n_s) begin
      bit_count <= 0;   // CS 해제: 처음부터
    end else if (sck_rise) begin
      if (bit_count == 3'd7) begin
        data <= {shift, mosi_s};  // 8번째 비트
        valid <= 1;
        bit_count <= 0;
      end else begin
        shift <= {shift[5:0], mosi_s};
        bit_count <= bit_count + 1'b1;
      end
    end
  end
end
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/19a037b/lab7/lab7_01_spi_rx/src/spi_slave_rx.v#L36-L60

CS가 LOW인 동안 상승 에지마다 MOSI를 시프트 레지스터 오른쪽에 넣으므로 먼저 온 bit7이 왼쪽 끝으로 밀려 올라간다. 7비트를 모은 뒤 8번째 에지에서 {shift, mosi_s}로 한 바이트를 만들고 valid를 한 클록 낸다. CS가 HIGH가 되면 bit_count를 0으로 돌리므로, 중간에 끊긴 바이트가 있어도 다음 바이트는 처음부터 받는다.

받은 값은 hex_line_tx가 MCU 출력과 같은 모양의 "RX=55" + CR LF 7글자로 바꿔 uart_tx(LAB3-07과 같은 9600 8N1)로 보낸다. 16진수 한 자리는 0~9면 0x30을, A~F면 0x37을 더해 ASCII로 만든다. 한 줄은 7.3 ms라 1초 간격의 바이트를 놓치지 않는다. 7.3 ms 안에 바이트가 또 오면 그 줄은 건너뛰지만 LED에는 표시된다.

파일	역할
src/spi_slave_rx.v	동기화, 에지 검출, 8비트 수신
src/hex_line_tx.v	바이트를 "RX=55" 줄로 UART 출력
src/uart_tx.v	UART 송신 8N1(LAB3-07)
src/spi_rx_uart.v	설계 top. LED = 마지막 바이트
sim/tb.sv	MCU 호스트 출력 파일로 검사
sim/tb_view.sv	파형 확인용. 0x55, 0xA5
constraints/spi_rx_uart.xdc	BDIO_0~2 입력, F6, LED

시뮬레이션 결과

tb.sv는 MCU 호스트 시뮬레이션이 보낸 8바이트(sim/mcu_spi.hex)를 MCU와 같은 781.25 kHz SPI 신호로 넣는다. CS를 내린 뒤 첫 에지까지 3 µs, 마지막 에지 뒤 CS를 올리기까지 2 µs를 두었다. UART는 실제 9600 bps로 받아 줄마다 비교한다. 마지막에는 4비트만 보내고 CS를 올린 뒤 0x3C를 보내, 끊긴 바이트 뒤에도 0x3C를 받는지 본다. 바이트 간격만 1초 대신 10 ms로 줄였다.

```
t=  7.272 ms  line 0: "RX=55\r\n"  LED=55
t= 17.286 ms  line 1: "RX=A5\r\n"  LED=A5
t= 27.301 ms  line 2: "RX=00\r\n"  LED=00
t= 37.315 ms  line 3: "RX=FF\r\n"  LED=FF
t= 47.330 ms  line 4: "RX=55\r\n"  LED=55
t= 57.345 ms  line 5: "RX=A5\r\n"  LED=A5
t= 67.359 ms  line 6: "RX=00\r\n"  LED=00
t= 77.374 ms  line 7: "RX=FF\r\n"  LED=FF
t= 87.448 ms  line 8: "RX=3C\r\n"  LED=3C
[Icarus·Vivado] PASS: 9 bytes, UART lines and LED match,
        partial-byte recovery 0x3C
```

로그:
https://github.com/tae2yi/UOS_ece2/blob/19a037b/lab7/report/pre/logs/lab7_01_spi_rx_tb_icarus.txt

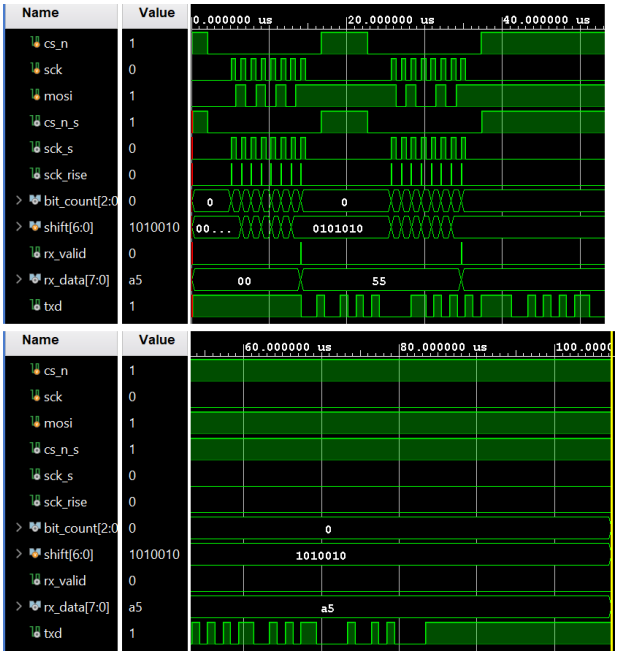


그림 4. FPGA 수신 Vivado 파형(tb_view, UART 1 Mbps). sck_rise마다 bit_count가 0→7로 늘고 8번째 에지에서 rx_valid와 rx_data(55, a5)가 나온 뒤 txd로 "RX=" 줄이 나간다.

cs_n_s와 sck_s는 원래 신호보다 2클록(40 ns) 늦다. shift가 0101010까지 모인 상태에서 8번째 에지가 오면 rx_data가 55가 되고, CS가 HIGH인 동안 bit_count는 0에 머문다. 두 번째 바이트 0xA5도 같은 방법으로 받아 rx_data가 a5로 바뀐다.

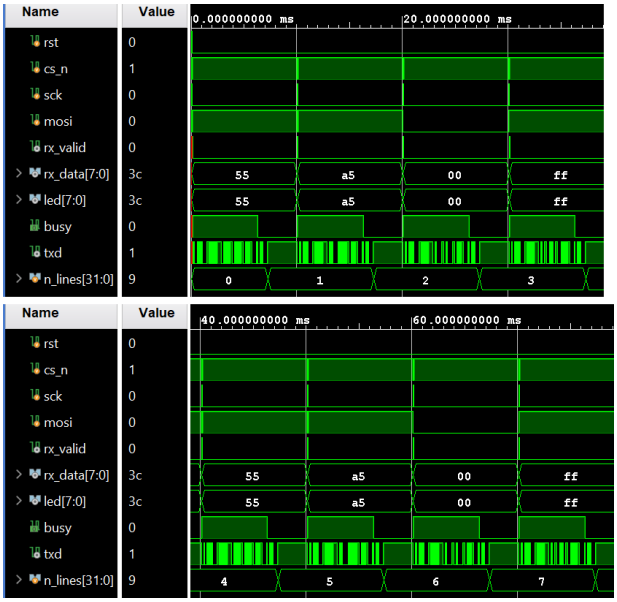


그림 5. FPGA 수신 Vivado 전체 파형(tb, 약 90 ms). 10 ms마다 바이트가 들어와 rx_valid 뒤 7.3 ms 동안 txd로 한 줄이 나가고, n_lines가 9까지 는다.

구현 결과와 보드에서 확인할 항목

신호	핀	연결
spi_sck	F8 (BDIO_0)	MCU PB3, 풀다운
spi_mosi	E8 (BDIO_1)	MCU PA7, 풀다운
spi_cs_n	A7 (BDIO_2)	MCU PB6, 풀업
uart_txd	F6	USB-UART → PC
led[7:0]	L4 M4 M2 N7 M7 M3 M1 N5	LED1(bit7)~LED8

입력에는 내부 풀업·풀다운을 켜서 선이 빠져도 SCK LOW, CS HIGH(선택 안 됨)로 보이게 했다. 잡음이 에지로 읽혀 엉뚱한 바이트가 출력되는 것을 막는다.

항목	값
LUT · 레지스터	88개 · 99개
타이밍	50 MHz에서 WNS +14.349 ns, 위반 없음
DRC	오류 0, 경고 1 (CFGBVS 미설정)
bitstream	lab7_01_spi_rx/bitstream/spi_rx_uart.bit

XDC:
https://github.com/tae2yi/UOS_ece2/blob/19a037b/lab7/lab7_01_spi_rx/constraints/spi_rx_uart.xdc

확인 항목	예상 관찰
준비	SW5 켬, 클록 스위치 F(50 MHz), SW1 리셋
FPGA COM(9600)	1초마다 RX=55, A5, 00, FF
MCU VCP	TX=값과 FPGA 출력이 같은 순서
LED (0x55)	LED1 꺼짐부터 꺼짐·켜짐 번갈아
LED (0xA5)	켜짐 꺼짐 켜짐 꺼짐 꺼짐 켜짐 꺼짐 켜짐
CS 선 제거	출력 멈춤(풀업으로 선택 안 됨)

STM32_3 과제11 FPGA → MCU 송신 구현

목적과 설계

FPGA가 마스터가 되어 SCK, MOSI, CS를 직접 만든다. 1초마다 0x55, 0xA5, 0x00, 0xFF를 차례로 보내 MCU 송신 예제와 같은 값을 비교할 수 있게 했다. DIP 스위치로 값을 고르는 방법도 생각했지만, BDIO 연결에 DIP 스위치 SW5를 쓰므로 혼동을 피하려고 정해진 순서로 보낸다. 보낸 값은 FPGA UART로 "TX=55"처럼 PC에 알리고 LED에도 표시한다.

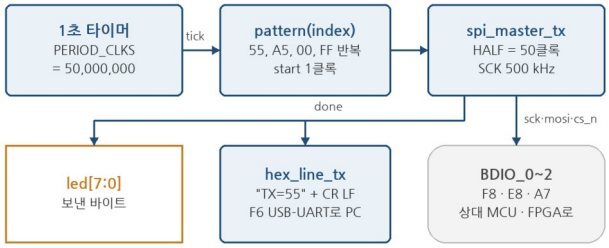


그림 6. FPGA 송신 회로(spi_tx_uart). 1초 타이머가 다음 값을 골라 spi_master_tx를 시작하고, 끝나면(done) UART와 LED에 표시한다.

spi_master_tx는 반주기(HALF = 50클록 = 1 µs)마다 한 단계씩 움직이는 상태 기계다. CS를 내리면서 bit7을 MOSI에 내놓고(LEAD), SCK를 올렸다 내리기를 8번 반복하며 내릴 때마다 다음 비트를 내놓는다(HIGH, LOW). 8번째 하강 에지 뒤 반주기 기다려 CS를 올리고(TRAIL), 다시 반주기 쉰다(GAP). SCK·MOSI·CS 모두 플립플롭에서 바로 나가므로 조합 논리의 글리치가 선에 실리지 않는다.

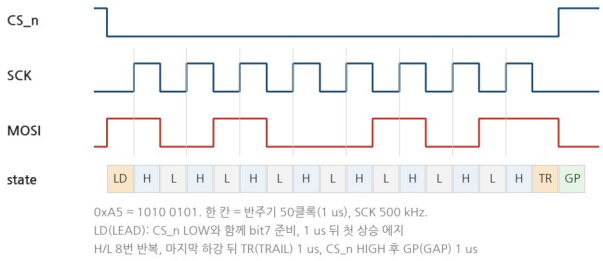


그림 7. 0xA5를 보낼 때 spi_master_tx의 상태 순서. CS LOW는 LEAD 1 + 비트 15반주기 + TRAIL 1 = 17 µs다.

핵심 코드

```
always @(posedge clk) begin
  if (rst_p) begin
    state <= IDLE; shift <= 0; bit_index <= 0; count <= 0;
    sck <= 0; mosi <= 0; cs_n <= 1; done <= 0;
  end else begin
    done <= 0;
    if (state != IDLE && !tick) count <= count - 1'b1;
    case (state)
      IDLE: if (start) begin
        shift <= data; mosi <= data[7]; cs_n <= 0;
        bit_index <= 0; count <= HALF - 1; state <= LEAD;
      end
      LEAD: if (tick) begin
        sck <= 1; count <= HALF - 1; state <= HIGH;
      end
      HIGH: if (tick) begin
        sck <= 0; count <= HALF - 1;
        if (bit_index == 3'd7) state <= TRAIL;
        else begin
          shift <= {shift[6:0], 1'b0};
          mosi <= shift[6];  // 다음 비트
          bit_index <= bit_index + 1'b1;
          state <= LOW;
        end
      end
      LOW: if (tick) begin
        sck <= 1; count <= HALF - 1; state <= HIGH;
      end
      TRAIL: if (tick) begin
        cs_n <= 1; mosi <= 0;
        count <= HALF - 1; state <= GAP;
      end
      GAP: if (tick) begin
        done <= 1; state <= IDLE;
      end
      default: state <= IDLE;
    endcase
  end
end
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/19a037b/lab7/lab7_02_spi_tx/src/spi_master_tx.v#L33-L71

count는 상태에 들어갈 때 HALF−1로 두고 0이 될 때까지 줄이므로 각 상태는 정확히 50클록이다. HIGH에서 SCK를 내리는 클록에 mosi <= shift[6]으로 다음 비트를 내고 shift를 왼쪽으로 민다. 수신 측은 그 뒤 반주기(1 µs) 동안 안정된 값을 상승 에지에서 읽는다.

파일	역할
src/spi_master_tx.v	Mode 0 마스터 상태 기계
src/spi_tx_uart.v	설계 top. 1초 타이머, 값 순서
src/hex_line_tx.v, uart_tx.v	"TX=55" 줄 출력(01과 같음)
sim/tb.sv	SPI 수신 모델로 타이밍·값 검사
sim/tb_view.sv	파형 확인용. 0x55, 0xA5
constraints/spi_tx_uart.xdc	BDIO_0~2 출력, F6, LED

시뮬레이션 결과

tb.sv 안에 오실로스코프와 MCU 수신 측 역할을 하는 SPI 수신 모델을 두었다. 바이트마다 CS LOW 동안 상승 에지가 정확히 8번인지, SCK 주기가 2000 ns인지, CS↓에서 첫 상승 에지까지 1000 ns인지, SCK가 HIGH인 동안 MOSI가 바뀌지 않는지, 읽은 값이 순서대로 맞는지 검사한다. 송신 간격만 10 ms로 줄였다. 읽은 바이트는 파일로 남겨 MCU 수신 호스트 시뮬레이션에 넣었다.

```
t= 10.000 ms  SPI byte 0: 55  8 edges OK
t= 20.000 ms  SPI byte 1: a5  8 edges OK
t= 30.000 ms  SPI byte 2: 00  8 edges OK
t= 40.000 ms  SPI byte 3: ff  8 edges OK
...  (8 bytes)
[Icarus·Vivado] PASS: SCK 500 kHz, Mode 0 timing,
        UART TX= lines match
$ slave < fpga_tx_bytes.hex   (MCU 수신 VCP)
RX=55 RX=A5 RX=00 RX=FF RX=55 RX=A5 RX=00 RX=FF
```

로그:
https://github.com/tae2yi/UOS_ece2/blob/19a037b/lab7/report/pre/logs/lab7_02_spi_tx_tb_icarus.txt

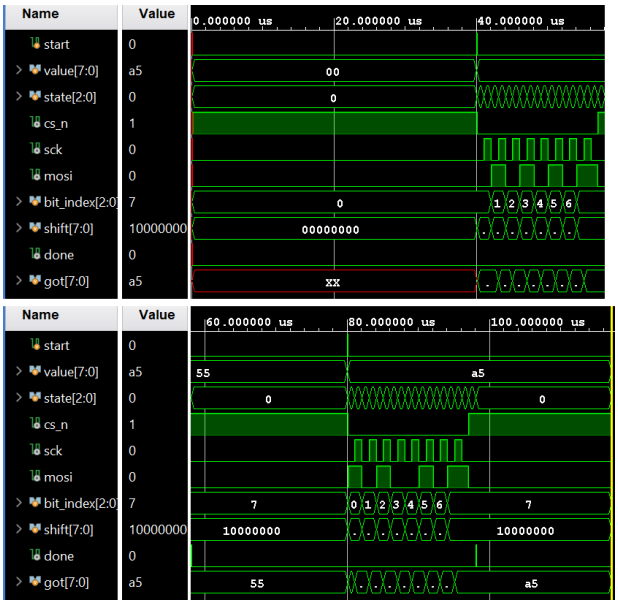


그림 8. FPGA 송신 Vivado 파형(tb_view, 송신 간격 40 µs). start 뒤 state가 1→2·3 반복→4→5로 가며 SCK 8주기가 나오고, TB가 상승 에지에서 모은 got이 55, a5가 된다.

cs_n이 내려가는 순간 mosi가 이미 bit7 값이고(0x55면 0, 0xA5면 1), 첫 SCK 상승 에지는 1 µs 뒤다. mosi는 SCK 하강과 같은 시각에만 바뀐다. bit_index가 7이 된 뒤 TRAIL·GAP을 지나 done이 한 클록 나오고, 그때 value가 LED와 UART로 간다.

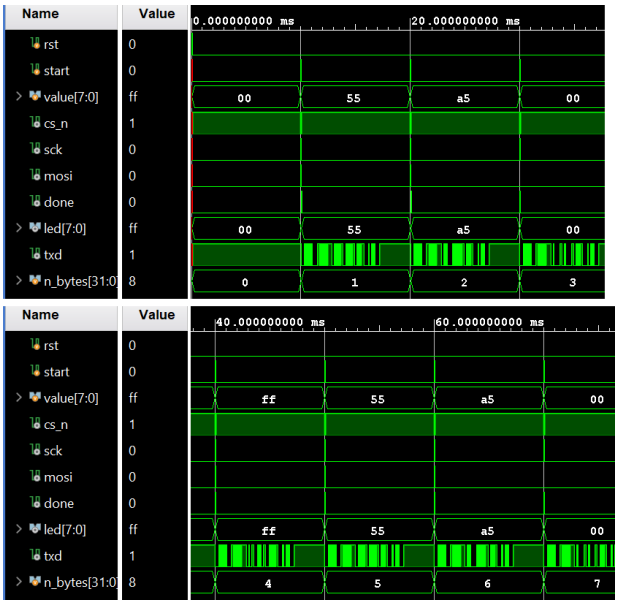


그림 9. FPGA 송신 Vivado 전체 파형(tb, 약 87 ms). 10 ms마다 value가 55·a5·00·ff로 바뀌며 한 바이트씩 나가고 txd로 "TX=" 줄이 뒤따른다.

구현 결과와 보드에서 확인할 항목

항목	값
LUT · 레지스터	116개 · 125개
타이밍	50 MHz에서 WNS +15.484 ns, 위반 없음
DRC	오류 0, 경고 1 (CFGBVS 미설정)
핀	F8 SCK, E8 MOSI, A7 CS_n (수신 설계와 같음)
bitstream	lab7_02_spi_tx/bitstream/spi_tx_uart.bit

XDC:
https://github.com/tae2yi/UOS_ece2/blob/19a037b/lab7/lab7_02_spi_tx/constraints/spi_tx_uart.xdc

매뉴얼대로 MCU에 잇기 전에 오실로스코프로 먼저 본다. 그다음 MCU에 수신 프로그램(-DSTM32_3_ROLE=2)을 올리고 나서 선을 잇는다.

확인 항목	예상 관찰
SCK	주기 2 µs(500 kHz), 바이트마다 8주기
CS_n	1초마다 17 µs LOW
CS↓ → 첫 SCK↑	1 µs, 이때 MOSI = bit7
FPGA COM	1초마다 TX=55, A5, 00, FF
MCU VCP	RX=값이 FPGA의 TX=와 같음
클록 스위치 F 아님	SCK·간격이 같은 비율로 느려짐

STM32_3 과제12 FPGA → FPGA 송수신 확인

지정된 두 조가 FPGA를 하나씩 맡아, 한쪽에 과제11의 송신 bitstream, 다른 쪽에 과제10의 수신 bitstream을 올린다. 두 설계의 핀이 같으므로 BDIO_0끼리, BDIO_1끼리, BDIO_2끼리, GND끼리 잇는다(그림 2). 수신 FPGA의 UART에 나오는 RX=값이 송신 FPGA의 TX=값과 같은지 본다. 역할 교대는 두 보드의 bitstream을 바꿔 올리는 것이며 선은 그대로 둔다.

두 보드는 각자의 50 MHz 발진기를 쓰므로 클록 주파수와 위상이 조금씩 다르다. 수신 회로가 SCK를 입력으로 받아 자기 클록에 동기화하는 구조라 이 차이는 문제가 되지 않아야 한다. 이를 확인하려고 lab7_03_spi_link에 두 설계를 함께 넣은 테스트벤치를 만들었다. 보드 A 클록은 20.000 ns, 보드 B는 20.050 ns(0.25 % 느림, 위상 7 ns 어긋남)이고, 선 지연으로 SCK 3 ns, MOSI 5 ns, CS 4 ns를 더했다. 두 UART 출력을 각각 PC 쪽에서 받아 비교한다.

```
t= 17.270 ms  A: "TX=55"  B: "RX=55"  B LED=55
t= 27.270 ms  A: "TX=A5"  B: "RX=A5"  B LED=A5
t= 37.270 ms  A: "TX=00"  B: "RX=00"  B LED=00
t= 47.270 ms  A: "TX=FF"  B: "RX=FF"  B LED=FF
...  (8 bytes)
[Icarus·Vivado] PASS: board A TX = board B RX
        (separate clocks, wire delay)
```

로그:
https://github.com/tae2yi/UOS_ece2/blob/19a037b/lab7/report/pre/logs/lab7_03_spi_link_tb_icarus.txt

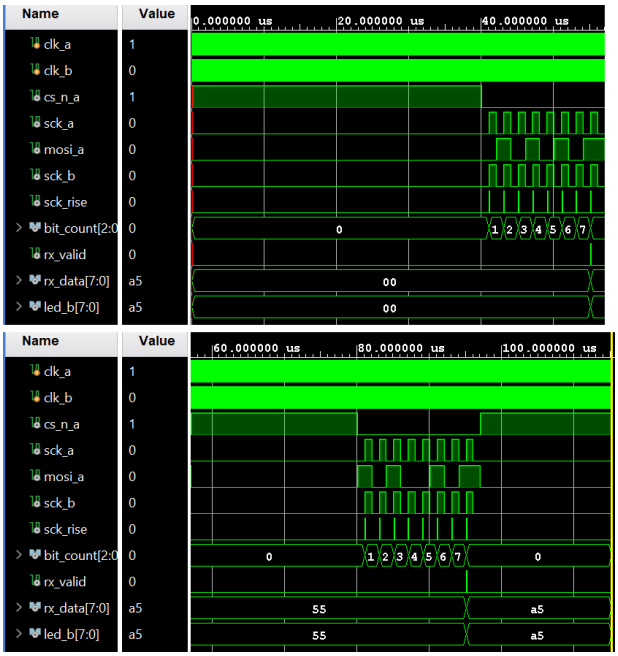


그림 10. FPGA-FPGA Vivado 파형(tb_view). 보드 A의 sck_a가 선 지연 뒤 sck_b로 보드 B에 들어가고, B 안에서 sck_rise 8번 뒤 rx_data와 led_b가 55, a5가 된다.

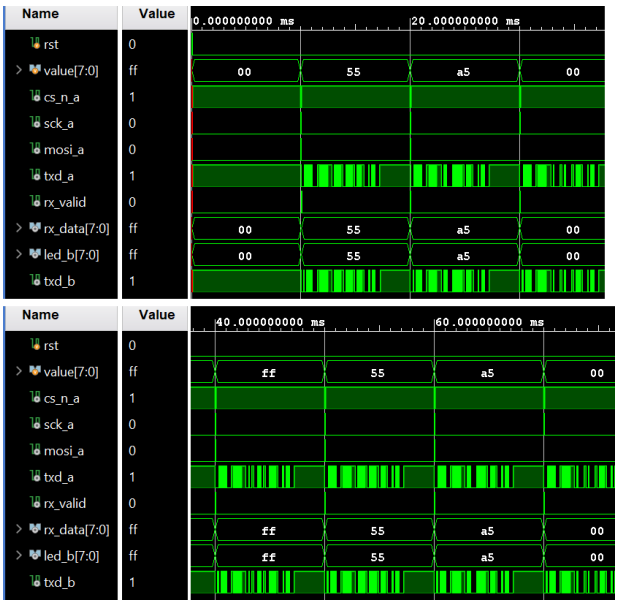


그림 11. FPGA-FPGA Vivado 전체 파형(tb). A가 10 ms마다 보낸 값을 B가 받아 rx_valid 뒤 txd_b로 "RX=" 줄을 내고, txd_a의 "TX=" 줄과 값이 같다.

확인 항목	예상 관찰
배선 전	두 보드 bitstream 확인(송신 1, 수신 1)
양쪽 SW5, GND	BDIO 연결 켬, GND 공통
수신 FPGA COM	RX=55, A5, 00, FF 반복
송신 FPGA COM	TX= 같은 순서(두 PC 화면 비교)
역할 교대	bitstream만 바꿔 같은 결과

4 예상 결과 정리와 실험 계획

이번 주의 공통점은 같은 바이트를 서로 다른 장치가 같은 규칙으로 읽는 것이다. Mode 0에서는 보내는 쪽이 하강 에지에 비트를 바꾸고 받는 쪽이 상승 에지에 읽으므로, SCK 반주기만큼의 여유가 생긴다. MCU는 SPI 하드웨어가 이를 처리하고, FPGA는 같은 규칙을 상태 기계(송신)와 동기화·에지 검출(수신)로 직접 만든다. 어느 연결이든 받는 쪽 출력(RX=)이 보낸 쪽 출력(TX=)과 같은지로 판단한다.

순서	할 일	기록할 것
1	과제8 기본 빌드 다운로드	VCP 화면, SCK·MOSI·CS 파형
2	0x55 고정·주기 측정	SCK 주기, CS↓→SCK 시간
3	과제9 수신 먼저, 송신 나중	두 VCP 화면, 배선 사진
4	과제9 역할 교대	CS 선 변경, 결과 화면
5	spi_rx_uart.bit, 과제10	FPGA COM 화면, LED
6	spi_tx_uart.bit, 파형	SCK 2 µs, CS 17 µs
7	MCU 수신 후 연결, 과제11	MCU VCP, FPGA COM
8	과제12 두 보드, 교대	두 COM 화면, 배선 사진

실험 후 보고서에서는 오실로스코프로 잰 SCK 주기·CS 시간과 이 보고서의 계산값(1.28 µs, 2 µs, 17 µs)을 비교하고, MCU HAL이 CS와 SCK 사이에 넣는 시간을 적는다. 값이 밀리거나 틀린 경우가 있으면 그때의 연결·실행 순서와 함께 원인을 정리한다.

참고 문헌

[1] STMicroelectronics, STM32CubeG4 MCU Firmware Package (stm32g4xx_hal_spi.c). https://github.com/STMicroelectronics/STM32CubeG4

[2] Sarah L. Harris and David Harris, Digital Design and Computer Architecture: ARM Edition, Morgan Kaufmann, 2015. ISBN 978-0-12-800056-4.